# Bloque 1: arreglos y matrices

## Programación: tres problemas de juez, uno por hora

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

# Las tres horas

## Cada hora

Un repaso corto de lo que cuesta cada operación y, después, un problema de UVa resuelto entre todos: leer el enunciado, probar la idea más directa, medirla contra los límites, encontrar la buena, seguirla en una tabla y escribirla en Python y en C++.

| Hora | Repaso | Problema |
|---|---|---|
| 1 | lectura, lista y `vector` | UVa 11078, *Open Credit System* |
| 2 | recorridos y dos índices | UVa 507, *Jill Rides Again* |
| 3 | matrices | UVa 108, *Maximum Sum* |

## Código

Las soluciones están en `ejercicios/1/`, en Python y en C++, con sus entradas `.in` y salidas `.out`. C++ se compila con `g++ -std=c++17 -Wall`.

# Contenido

1 Hora 1. UVa 11078: Open Credit System

2 Hora 2. UVa 507: Jill Rides Again

3 Hora 3. UVa 108: Maximum Sum

4 Tarea del bloque

# Contenido

1 Hora 1. UVa 11078: Open Credit System

2 Hora 2. UVa 507: Jill Rides Again

3 Hora 3. UVa 108: Maximum Sum

4 Tarea del bloque

# ¿Qué trae esta entrada?

## Archivo A

```
5
3 -1 4 1 5
```

## Archivo B

```
5
3 -1
4 1
5
```

## Pregunta

Un programa que lee los valores de la segunda línea con `input().split()`, ¿lee también el archivo B?

# ¿Qué trae esta entrada?

## Respuesta

No: con B, la segunda línea trae solo `3 -1`. Se lee por palabras:

- Python: `datos = sys.stdin.read().split()` trae toda la entrada partida en espacios y saltos de línea por igual; un índice `pos` avanza sobre esa lista.
- C++: `scanf("%d", &x)` salta todo espacio en blanco antes del número, saltos de línea incluidos.

# Leer hasta fin de archivo

## Pregunta

La entrada trae pares de enteros y nadie dice cuántos. ¿Cuándo para el ciclo de lectura?

## Python: `hasta_eof.py`

```python
import sys

for linea in sys.stdin:
    partes = linea.split()
    if len(partes) == 2:
        a = int(partes[0])
        b = int(partes[1])
        print(a, b, a + b)
```

## C++: `hasta_eof.cpp`

```cpp
#include <cstdio>

int main() {
    int a, b;
    while (scanf("%d %d", &a, &b) ==
    ↪  2) {
        printf("%d %d %d\n", a, b, a
        ↪  + b);
    }
    return 0;
}
```

# Leer hasta fin de archivo

## Respuesta

Cuando se acaba la entrada. En Python, `for linea in sys.stdin` termina solo; en C++, `scanf` devuelve cuántos valores leyó y al final ya no devuelve 2. UVa 100, de los problemas del bloque, se lee así.

# Lo que cuesta cada operación

| Operación | Python (`list`) | C++ (`vector`) | Costo |
|---|---|---|---|
| Acceder por posición | `a[i]` | `a[i]` | $O(1)$ |
| Tamaño | `len(a)` | `a.size()` | $O(1)$ |
| Agregar al final | `a.append(x)` | `a.push_back(x)` | $O(1)$ en promedio |
| Quitar el último | `a.pop()` | `a.pop_back()` | $O(1)$ |
| Insertar en $p$ | `a.insert(p, x)` | `a.insert(a.begin() + p, x)` | $O(n-p)$ |
| Borrar en $p$ | `del a[p]` | `a.erase(a.begin() + p)` | $O(n-p)$ |
| Buscar un valor | `x in a` | recorrido con `for` | $O(n)$ |
| Máximo | `max(a)` | recorrido con `for` | $O(n)$ |

## El costo que no se ve

`x in a` y `max(a)` caben en una línea, pero recorren la lista. Dentro de un ciclo de $n$ vueltas, el programa cuesta $O(n^2)$. Lo mismo pasa con `a.insert(0, x)`: la `list` de Python es un arreglo, no una lista enlazada.

# UVa 11078: Open Credit System

## Enunciado, en corto

Un curso tiene $n$ estudiantes, numerados por antigüedad: si $i < j$, el estudiante $i$ es más antiguo que el $j$. Cada uno presentó una prueba. El profesor quiere la mayor diferencia que un antiguo le saca a uno más nuevo: el máximo de $a[i] - a[j]$ con $i < j$. Sin valor absoluto.

## Límites

Menos de 20 casos; en cada uno, $2 \leq n \leq 100\,000$, un puntaje por línea, $|a[i]| < 150\,000$.

## Direcciones

Enunciado: `https://onlinejudge.org/external/110/11078.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2019`

# ¿Entendimos el problema?

## Muestra

```
3
2
100
20
4
4 3 2 1
4
1 2 3 4
```

En el archivo, un número por línea.

## Entradas y salidas

Entra $T$ y, por caso, $n$ y los $n$ puntajes. Sale un número por caso.

## A mano

- $[100, 20]$: una sola pareja, $100 - 20 = 80$.
- $[4, 3, 2, 1]$: la mejor es $4 - 1 = 3$.
- $[1, 2, 3, 4]$: todas las parejas dan negativo; la mejor es $1 - 2 = -1$.

## Pregunta

En el tercer caso, ¿por qué la respuesta no es 3, que es $4 - 1$?

# ¿Entendimos el problema?

## Respuesta

Porque el 4 es el último: es el más nuevo y tiene que ir a la derecha en la resta. Con $i < j$, las parejas del tercer caso son $1-2$, $1-3$, $1-4$, $2-3$, $2-4$ y $3-4$, y la mayor vale $-1$.

## Lo que eso dice de la salida

La respuesta puede ser negativa. Un programa que imprime 0 cuando ningún antiguo supera a uno nuevo falla el tercer caso de la muestra.

# Primera idea: todas las parejas

## Python: `credito_ingenua.py`

```python
def mayor_diferencia(a):
    mejor = a[0] - a[1]
    for i in range(len(a)):
        for j in range(i + 1, len(a)):
            if a[i] - a[j] > mejor:
                mejor = a[i] - a[j]
    return mejor
```

## ¿Cuánto cuesta?

El ciclo de adentro da $n - 1 - i$ vueltas: en total $n(n-1)/2$ parejas. Con $n = 100\,000$ son unos $5 \cdot 10^9$ por caso, y puede haber 19 casos.

# Primera idea: todas las parejas

## Medido (un caso; Python 3.14, Ryzen 7 7435HS)

| $n$ | dos ciclos | un recorrido |
|---:|---:|---:|
| 10 000 | 2,84 s | 0,02 s |
| 20 000 | 11,32 s | 0,02 s |

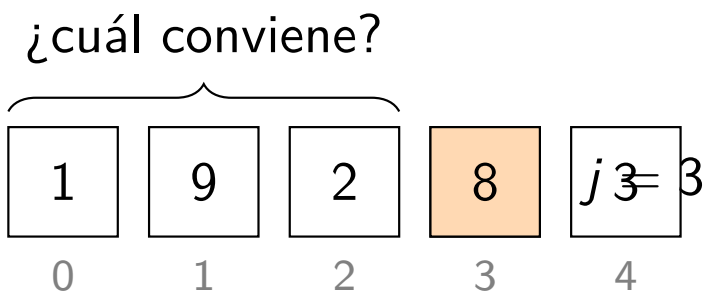
## Contra los límites

Al duplicar $n$ el tiempo de los dos ciclos se multiplica por 4. De 10 000 a 100 000 el factor es $10^2 = 100$: unos 280 segundos por caso. El límite del juez es de 3 segundos para todo el archivo.

# Fijar al más nuevo

## Pregunta

Fije el estudiante nuevo $j$. De todos los antiguos $i < j$, ¿cuál le saca la mayor diferencia?



# Fijar al más nuevo

## Respuesta

El de mayor puntaje entre $a[0], \ldots, a[j-1]$: restar siempre el mismo $a[j]$ deja la diferencia más grande cuando el primer término es el mayor. Para $j = 3$, el 9: $9 - 8 = 1$.

## Y ese mayor no hay que buscarlo

Al pasar de $j$ a $j+1$ entra un solo candidato nuevo, $a[j]$. El mayor de los antiguos se actualiza con una comparación, como el máximo de un recorrido. La respuesta es el mejor $\mathit{mayor} - a[j]$ que aparezca en el camino.

## Costo

Un recorrido, dos comparaciones por vuelta: $\Theta(n)$ por caso.

# Traza con cinco estudiantes

## Estado

`mayor` es el máximo de $a[0..j-1]$ y `mejor` la mejor diferencia vista. Con $a = [1, 9, 2, 8, 3]$ arrancan en $a[0] = 1$ y en $a[0] - a[1] = -8$.

| $j$ | $a[j]$ | $mayor - a[j]$ | `mejor` | `mayor` al salir |
|---|---|---|---|---|
| inicio | | | $-8$ | 1 |
| 1 | 9 | $1 - 9 = -8$ | $-8$ | 9 |
| 2 | 2 | ? | ? | ? |
| 3 | 8 | ? | ? | ? |
| 4 | 3 | ? | ? | ? |

# Traza con cinco estudiantes

## Estado

$a = [1, 9, 2, 8, 3]$.

| $j$ | $a[j]$ | $mayor - a[j]$ | `mejor` | `mayor` al salir |
|---|---|---|---|---|
| inicio | | | $-8$ | 1 |
| 1 | 9 | $1 - 9 = -8$ | $-8$ | 9 |
| 2 | 2 | $9 - 2 = 7$ | 7 | 9 |
| 3 | 8 | $9 - 8 = 1$ | 7 | 9 |
| 4 | 3 | $9 - 3 = 6$ | 7 | 9 |

## Respuesta: 7

El orden dentro de la vuelta importa: primero se compara $mayor - a[j]$, después $a[j]$ entra al máximo. Al revés, en $j = 1$ se calcularía $9 - 9 = 0$, una pareja del estudiante consigo mismo.

# Python: open-credit-system.py

## El recorrido

```
def mayor_diferencia(a):
    mayor = a[0]
    mejor = a[0] - a[1]
    for j in range(1, len(a)):
        if mayor - a[j] > mejor:
            mejor = mayor - a[j]
        if a[j] > mayor:
            mayor = a[j]
    return mejor
```

# Python: open-credit-system.py

## La lectura por palabras

```
def main():
    datos = sys.stdin.read().split()
    t = int(datos[0])
    pos = 1
    for caso in range(t):
        n = int(datos[pos])
        a = []
        for k in range(n):
            a.append(int(datos[pos + 1 + k]))
        pos = pos + 1 + n
        print(mayor_diferencia(a))


main()
```

# C++: open-credit-system.cpp

## El recorrido

```cpp
int mayor_diferencia(const vector<int>& a) {
    int mayor = a[0];
    int mejor = a[0] - a[1];
    for (int j = 1; j < (int) a.size(); j++) {
        if (mayor - a[j] > mejor) {
            mejor = mayor - a[j];
        }
        if (a[j] > mayor) {
            mayor = a[j];
        }
    }
    return mejor;
}
```

# C++: open-credit-system.cpp

## La lectura

```cpp
int main() {
    int t;
    scanf("%d", &t);
    for (int caso = 0; caso < t; caso++) {
        int n;
        scanf("%d", &n);
        vector<int> a(n);
        for (int k = 0; k < n; k++) {
            scanf("%d", &a[k]);
        }
        printf("%d\n", mayor_diferencia(a));
    }
    return 0;
}
```

## ¿Cabe en int?

$|a[i]| < 150\,000$, así que la diferencia está entre $-300\,000$ y $300\,000$.
Sobra espacio en 32 bits.

# Trampas

## `mejor = 0` al comienzo: pasa dos casos de la muestra y falla el tercero

| $j$ | $a[j]$ | $mayor - a[j]$ | `mejor` | `mayor` al salir |
|---|---|---|---|---|
| inicio | | | 0 | 1 |
| 1 | 2 | $-1$ | 0 | 2 |
| 2 | 3 | $-1$ | 0 | 3 |
| 3 | 4 | $-1$ | 0 | 4 |

## Corrección

`mejor` arranca con una pareja que existe, $a[0] - a[1]$. Así el resultado siempre es la diferencia de dos estudiantes, aunque sea negativa.

## `max(a[:j])` dentro del ciclo

Vuelve a dar $O(n^2)$: es el costo que no se ve de la tabla del repaso.

# Variante para pensar

## Con distancia mínima

El profesor solo compara estudiantes que estén al menos $d$ puestos aparte: el máximo de $a[i] - a[j]$ con $j - i \geq d$. Con $a = [1, 9, 2, 8, 3]$ y $d = 2$, la respuesta es $9 - 3 = 6$. ¿Sigue siendo un solo recorrido?

## Pista

Para el $j$ actual, los antiguos permitidos son $a[0], \ldots, a[j - d]$. Cuando $j$ avanza una posición, entra un solo candidato, $a[j - d]$.

# Contenido

1 Hora 1. UVa 11078: Open Credit System

2 Hora 2. UVa 507: Jill Rides Again

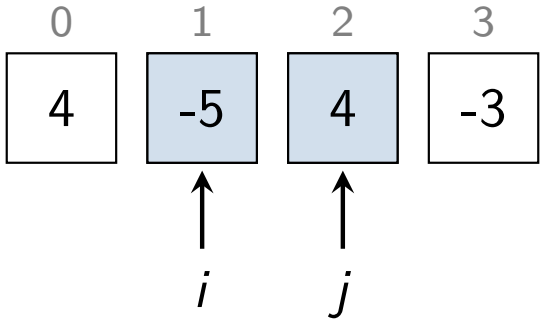
3 Hora 3. UVa 108: Maximum Sum

4 Tarea del bloque

# ¿Cuántos tramos tiene un arreglo?

## Pregunta

Un tramo de $v$ es un bloque de casillas seguidas, $v[i..j]$ con $i \leq j$.
¿Cuántos tramos tiene un arreglo de 4 casillas?




## Respuesta

Diez: cuatro empiezan en 0, tres en 1, dos en 2 y uno en 3. Con $n$ casillas son $n + (n-1) + \cdots + 1 = n(n+1)/2$, del orden de $n^2$. Un tramo queda dicho por dos índices, el primero y el último.

# La suma de un tramo sin recorrerlo

## Pregunta

Sumar $v[i..j]$ recorriéndolo cuesta $j - i + 1$ pasos. ¿Se puede en $O(1)$?

## Respuesta: los acumulados

Con $acum[0] = 0$ y $acum[k+1] = acum[k] + v[k]$, la suma $v[i] + \cdots + v[j]$ es $acum[j+1] - acum[i]$. Prepararlos es un recorrido, $\Theta(n)$, una sola vez.

## Con $v = [4, -5, 4, -3]$

| $k$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $v[k]$ | 4 | $-5$ | 4 | $-3$ | |
| $acum[k]$ | 0 | 4 | $-1$ | 3 | 0 |

$$v[1.,2] = -5 + 4 = acum[3] - acum[1] = 3 - 4 = -1.$$

# UVa 507: Jill Rides Again

## Enunciado, en corto

Una ruta de bus tiene $s$ paradas. Entre la parada $i$ y la $i+1$ hay una calle con un valor $n_i$, distinto de cero, que dice cuánto le gusta a Jill. Ella baja en una parada, pedalea y vuelve a subir en otra. Hay $b$ rutas, cada una con $2 \leq s \leq 20\,000$ paradas. Se pide el tramo de paradas $i < j$ con la mayor suma $n_i + \cdots + n_{j-1}$.

## Desempates y salida

Si varios tramos tienen la suma máxima, gana el más largo (mayor $j - i$); si siguen empatados, el que empieza antes (menor $i$). Se imprime `The nicest part of route r is between stops i and j`, o `Route r has no nice parts` si la suma máxima no es positiva.

# UVa 507: Jill Rides Again

## Direcciones

Enunciado: `https://onlinejudge.org/external/5/507.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=448`

# ¿Entendimos el problema?

## Muestra

```
3
3
-1 6
10
4 -5 4 -3 4
4 -4 4 -5
4
-2 -3 -4
```

En el archivo, un número por línea.

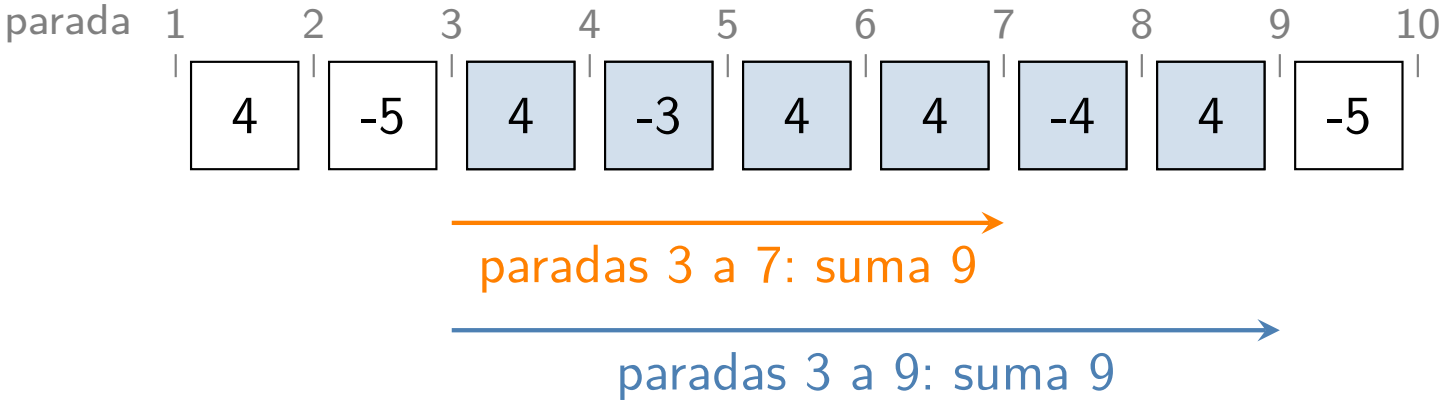
## A mano

- Ruta 1: calles $-1$ y 6. Lo mejor es la segunda calle sola, de la parada 2 a la 3.
- Ruta 3: todas negativas. No hay tramo positivo: `Route 3 has no nice parts`.

## Pregunta

En la ruta 2, el tramo de las paradas 3 a 7 suma $4 - 3 + 4 + 4 = 9$. ¿Por qué la respuesta es de la 3 a la 9?

# ¿Entendimos el problema?




## Respuesta

Los dos suman 9 y el de 3 a 9 es más largo: las dos calles que agrega, $-4$ y 4, suman cero. Es el primer desempate.

## Índices

`v[k]` une las paradas $k+1$ y $k+2$, así que el tramo de calles $v[i..j]$ va de la parada $i+1$ a la $j+2$.

# Primera idea: todos los tramos

## Python: `jill_acumulados.py`, con la suma de cada tramo en $O(1)$

```python
def mejor_tramo(v):
    acum = [0]
    for x in v:
        acum.append(acum[-1] + x)
    mejor = 0
    mi = 0
    mj = -1
    for i in range(len(v)):
        for j in range(i, len(v)):
            suma = acum[j + 1] - acum[i]
            if suma > mejor or (suma == mejor and j - i > mj - mi):
                mejor = suma
                mi = i
                mj = j
    return mejor, mi, mj
```

El desempate sale del orden: $i$ crece afuera y la comparación de largo es estricta, así que entre dos tramos iguales en suma y en largo queda el de menor $i$.

# Primera idea: todos los tramos

## ¿Cuánto cuesta?

Los tramos son $(s-1)s/2$, unos $2 \cdot 10^8$ con $s = 20\,000$. Sin acumulados, cada suma cuesta además hasta $s$ pasos y el total pasa a $O(s^3)$.

## Medido (una ruta; Python 3.14, Ryzen 7 7435HS)

| $s$ | todos los tramos | un recorrido |
|---:|---:|---:|
| 5 000 | 1,29 s | 0,02 s |
| 10 000 | 5,11 s | 0,02 s |
| 20 000 | 20,77 s | 0,02 s |

## Contra los límites

Veinte segundos por una sola ruta de tamaño máximo, contra 3 segundos de límite para el archivo entero, que trae varias.

# ¿Sirve lo que se lleva sumado?

**Pregunta**

Se recorre la ruta llevando la suma del tramo que empezó en `inicio` y termina en la calle actual $k$. Si esa suma se vuelve negativa, ¿le conviene a algún tramo que siga después de $k$ arrastrarla?

**Con $v = [4, -5, 4, \ldots]$**

Después de $4 - 5$ la suma es $-1$. Un tramo que empiece en la calle 2 vale lo mismo que con ese $-1$ delante, más uno.

# ¿Sirve lo que se lleva sumado?

## Respuesta

No. Un prefijo negativo solo resta: la suma vuelve a 0 y el tramo empieza en $k + 1$. Es la idea de Kadane.

## El recorrido, en palabras: $\Theta(s)$ por ruta

1. Sumar $v[k]$ a la suma actual.
2. Si la suma supera a la mejor, o la iguala con un tramo más largo, guardar el tramo $[inicio, k]$.
3. Si la suma quedó negativa, volver a 0 y poner $inicio = k + 1$.

## ¿Negativa o no positiva?

Se reinicia con $suma < 0$, no con $suma \leq 0$. Un prefijo que suma cero no resta, pero alarga el tramo, y el enunciado premia el tramo largo.

# Traza con la ruta 2

| $k$ | $v[k]$ | `suma` | `inicio` | `mejor` | tramo $[mi, mj]$ |
|---|---|---|---|---|---|
| 0 | 4 | 4 | 0 | 4 | $[0, 0]$ |
| 1 | $-5$ | $-1 \to 0$ | $0 \to 2$ | 4 | $[0, 0]$ |
| 2 | 4 | 4 | 2 | 4 | ? |
| 3 | $-3$ | ? | ? | ? | ? |
| 4 | 4 | ? | ? | ? | ? |
| 5 | 4 | ? | ? | ? | ? |
| 6 | $-4$ | ? | ? | ? | ? |
| 7 | 4 | ? | ? | ? | ? |
| 8 | $-5$ | ? | ? | ? | ? |

**Pregunta**

En $k = 2$ la suma vuelve a 4, igual que la mejor. ¿Cambia el tramo guardado?

# Traza con la ruta 2

| $k$ | $v[k]$ | `suma` | `inicio` | `mejor` | tramo $[mi, mj]$ |
|---|---|---|---|---|---|
| 0 | 4 | 4 | 0 | 4 | $[0, 0]$ |
| 1 | $-5$ | $-1 \to 0$ | $0 \to 2$ | 4 | $[0, 0]$ |
| 2 | 4 | 4 | 2 | 4 | $[0, 0]$, mismo largo |
| 3 | $-3$ | 1 | 2 | 4 | $[0, 0]$ |
| 4 | 4 | 5 | 2 | 5 | $[2, 4]$ |
| 5 | 4 | 9 | 2 | 9 | $[2, 5]$ |
| 6 | $-4$ | 5 | 2 | 9 | $[2, 5]$ |
| 7 | 4 | 9 | 2 | 9 | $[2, 7]$, más largo |
| 8 | $-5$ | 4 | 2 | 9 | $[2, 7]$ |

**Respuesta**

En $k = 2$ no: el tramo $[2, 2]$ tiene el mismo largo que $[0, 0]$ y empieza después.
En $k = 7$ sí: suma 9 otra vez, con más calles. Calles $[2, 7]$ son las paradas 3 a 9.

# Python: jill-rides-again.py

## El recorrido

```
def mejor_tramo(v):
    # v[k] es la calle entre las paradas k + 1 y k + 2
    suma = 0
    inicio = 0
    mejor = 0
    mi = 0
    mj = -1
    for k in range(len(v)):
        suma = suma + v[k]
        if suma > mejor or (suma == mejor and k - inicio > mj - mi):
            mejor = suma
            mi = inicio
            mj = k
        if suma < 0:
            suma = 0
            inicio = k + 1
    return mejor, mi, mj
```

# Python: jill-rides-again.py

## La lectura y la salida

```
def main():
    datos = sys.stdin.read().split()
    b = int(datos[0])
    pos = 1
    for r in range(1, b + 1):
        s = int(datos[pos])
        v = []
        for k in range(s - 1):
            v.append(int(datos[pos + 1 + k]))
        pos = pos + s
        mejor, mi, mj = mejor_tramo(v)
        if mejor > 0:
            print("The nicest part of route %d is between stops %d and %d"
                  % (r, mi + 1, mj + 2))
        else:
            print("Route %d has no nice parts" % r)


main()
```

# C++: jill-rides-again.cpp

## El recorrido

```cpp
long long mejor_tramo(const vector<int>& v, int& mi, int& mj) {
    long long suma = 0;
    int inicio = 0;
    long long mejor = 0;
    mi = 0;
    mj = -1;
    for (int k = 0; k < (int) v.size(); k++) {
        suma = suma + v[k];
        if (suma > mejor || (suma == mejor && k - inicio > mj - mi)) {
            mejor = suma;
            mi = inicio;
            mj = k;
        }
        if (suma < 0) {
            suma = 0;
            inicio = k + 1;
        }
    }
    return mejor;
}
```

# C++: jill-rides-again.cpp

```cpp
int main() {
    int b;
    scanf("%d", &b);
    for (int r = 1; r <= b; r++) {
        int s;
        scanf("%d", &s);
        vector<int> v(s - 1);
        for (int k = 0; k < s - 1; k++) {
            scanf("%d", &v[k]);
        }
        int mi, mj;
        long long mejor = mejor_tramo(v, mi, mj);
        if (mejor > 0) {
            printf("The nicest part of route %d is between stops %d and %d\n",
                   r, mi + 1, mj + 2);
        } else {
            printf("Route %d has no nice parts\n", r);
        }
    }
    return 0;
}
```

# Trampas

## Reiniciar con *suma* $\leq 0$

Con $v = [1, -1, 5]$, los tramos $[0, 2]$ y $[2, 2]$ suman 5, y gana el largo: paradas 1 a 4. Si la suma se reinicia al llegar a cero, en $k = 1$ el tramo arranca de nuevo en la calle 2 y la respuesta sale de la parada 3 a la 4.

## Comparar solo la suma

Con `suma > mejor` y nada más, la ruta 2 de la muestra da paradas 3 a 7.

## La salida exacta

Paradas desde 1: la parada final es $mj + 2$. Cuando la mejor suma no es positiva va la otra frase, y la ruta se numera aunque no tenga tramo.

# Variante para pensar

## Ruta circular

Después de la última parada el bus vuelve a la primera, así que un tramo puede dar la vuelta: terminar en la calle 0 después de pasar por la última. ¿Cómo se halla la mejor suma con un recorrido?

## Pista

Un tramo que da la vuelta deja por fuera un tramo del medio. Lo que queda es la suma total menos ese tramo: hay que quitar el de suma mínima, que también se halla con un recorrido.

# Contenido

1 Hora 1. UVa 11078: Open Credit System

2 Hora 2. UVa 507: Jill Rides Again

3 Hora 3. UVa 108: Maximum Sum

4 Tarea del bloque

# ¿Cómo se guarda una tabla?

## Python: lista de listas

`m[i]` es la fila $i$; `m[i][j]`, la celda. Se construye fila por fila con `append`.

## C++: `vector<vector<int>>`

`vector<vector<int>> m(n, vector<int>(n));` crea $n$ filas de $n$ ceros.

## Lo que cuesta

`m[i][j]` es $O(1)$; recorrer la matriz de $f \times c$ es $\Theta(f \cdot c)$; sumar una fila es $\Theta(c)$ y una columna, $\Theta(f)$.

# ¿Cómo se guarda una tabla?

## Pregunta: ¿qué imprime?

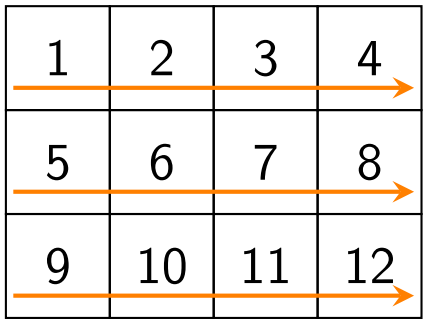
```
# Dos formas de crear una matriz de 3 x 3 con ceros.
mala = [[0] * 3] * 3
buena = [[0] * 3 for i in range(3)]
mala[0][0] = 7
buena[0][0] = 7
print("mala: ", mala)
print("buena:", buena)
```

## Respuesta

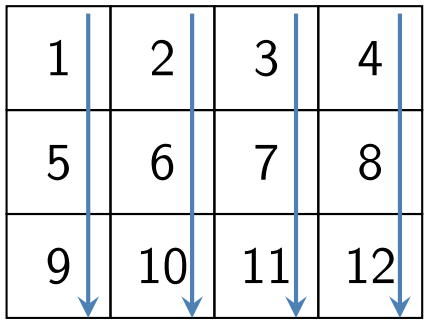
```
mala:  [[7, 0, 0], [7, 0, 0], [7, 0, 0]]
buena: [[7, 0, 0], [0, 0, 0], [0, 0, 0]]
```

`[fila] * 3` guarda tres referencias a la misma lista. Cada fila de una matriz se crea por separado.

# Filas, columnas y vecinas




*i* afuera




*j* afuera

## Qué ciclo va afuera

El de afuera se queda quieto mientras el de adentro corre. Para sumar una columna, *j* va afuera.

## Pregunta

¿Cuántas vecinas tiene una celda, contando lados y esquinas? ¿Y una del borde?

# Filas, columnas y vecinas

## Respuesta

Ocho en el interior, cinco en un borde, tres en una esquina. Son $(i + d_i, j + d_j)$ con $d_i, d_j \in \{-1, 0, 1\}$, sin $d_i = d_j = 0$, y solo cuentan si $0 \leq i + d_i < f$ y $0 \leq j + d_j < c$. En Python `m[-1]` no da error, da la última fila: el borde se revisa antes del acceso.

# Acumulados por columna

## Pregunta

$col[c]$ es la suma de la columna $c$ entre las filas `arriba` y `abajo`. Si `abajo` baja una fila, ¿hay que sumar todo otra vez?

## Respuesta

No: a cada columna se le suma la celda nueva, $col[c] \mathrel{+}= m[abajo][c]$. Cuesta $\Theta(n)$ por fila.

## Con la matriz de la muestra de UVa 108 y $arriba = 1$

| `abajo` | $col[0]$ | $col[1]$ | $col[2]$ | $col[3]$ |
|---|---|---|---|---|
| 1 | 9 | 2 | $-6$ | 2 |
| 2 | 5 | 3 | $-10$ | 3 |
| 3 | 4 | 11 | $-10$ | 1 |

# UVa 108: Maximum Sum

## Enunciado, en corto

Una matriz de $N \times N$ enteros, positivos y negativos. Se pide la mayor suma de un subrectángulo: un bloque de filas y columnas seguidas, de $1 \times 1$ en adelante.

## Límites

$N \leq 100$; cada valor en $[-127, 127]$. Tras $N$ vienen los $N^2$ números fila por fila, separados por espacios y saltos de línea en cualquier reparto.

## Direcciones

Enunciado: `https://onlinejudge.org/external/1/108.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=44`

# ¿Entendimos el problema?

## Muestra, tal como viene

```
4
0 -2 -7 0 9 2 -6          2
-4 1 -4 1 -1
8 0 -2
```

| | | | |
|---|---|---|---|
| 0 | -2 | -7 | 0 |
| 9 | 2 | -6 | 2 |
| -4 | 1 | -4 | 1 |
| -1 | 8 | 0 | -2 |

## A mano

Sale un número: el bloque marcado, filas 1 a 3 y columnas 0 y 1, suma $9 + 2 - 4 + 1 - 1 + 8 = 15$.

## Pregunta

La segunda línea trae 8 números y la última 3. ¿Qué dice eso de cómo leer?

## Respuesta

Que se leen $N^2$ números por palabras; la celda $(i, j)$ es el número $i \cdot N + j$.

# Primera idea: todos los rectángulos

## ¿Cuántos hay?

Un rectángulo es un par de filas $f_1 \leq f_2$ y un par de columnas $c_1 \leq c_2$: $(N(N+1)/2)^2$. Con $N = 100$, $5050^2 \approx 2{,}6 \cdot 10^7$.

## Sumando cada uno celda por celda

Cuatro ciclos para elegirlo y dos más para sumarlo: $O(N^6)$. Contando celdas, son $(N(N+1)(N+2)/6)^2 \approx 2{,}9 \cdot 10^{10}$ sumas. No pasa.

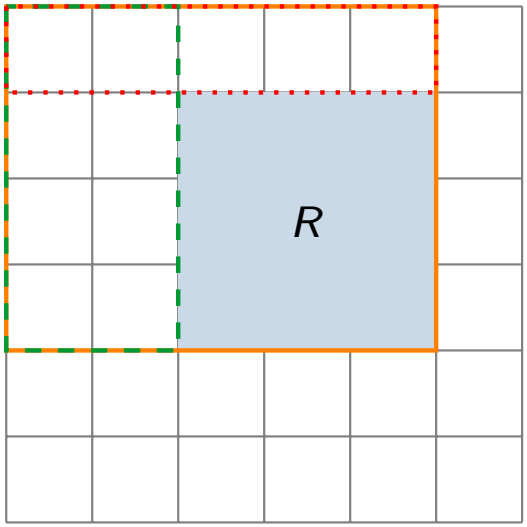
## Pregunta

En la hora 2, la suma de un tramo salió en $O(1)$ con acumulados. ¿Hay acumulados de un rectángulo?

# Acumulados en dos dimensiones

## Respuesta

Sí. $P[i][j]$ es la suma del rectángulo que va de la esquina $(0, 0)$ hasta la celda $(i-1, j-1)$, con $P$ en ceros en la fila 0 y la columna 0. Un rectángulo cualquiera sale de cuatro esquinas:




$$R = P[f_2{+}1][c_2{+}1] - P[f_1][c_2{+}1] - P[f_2{+}1][c_1] + P[f_1][c_1]$$

La esquina de arriba a la izquierda se resta dos veces, por eso vuelve a sumarse.

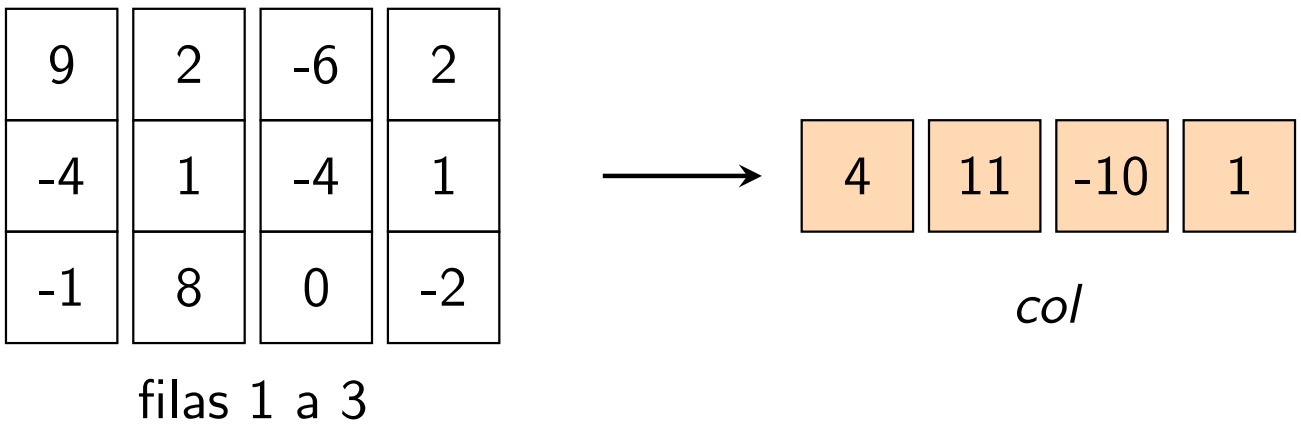
## Costo

$P$ se llena en $\Theta(N^2)$; cada rectángulo cuesta $O(1)$; todos, $O(N^4)$: unas $2{,}6 \cdot 10^7$ consultas, mil veces menos que antes.

# ¿Ya resolvimos esto?

## Pregunta

Fije las filas `arriba` y `abajo`. Cada rectángulo entre ellas es un tramo de columnas. Si cada columna se cambia por su suma entre esas filas, ¿qué problema queda?



# ¿Ya resolvimos esto?

## Respuesta

El de la hora 2: el tramo de suma máxima de *col*. Con $[4, 11, -10, 1]$ es $4 + 11 = 15$, el mismo rectángulo de la muestra.

## El algoritmo

1. Para cada `arriba`, poner *col* en ceros.
2. Para cada `abajo` desde `arriba`: sumar la fila `abajo` a *col*, como en el repaso, y pasar *col* por el recorrido de Kadane.
3. Quedarse con el mayor resultado.

## Costo

$N(N+1)/2$ pares de filas, y por cada uno $\Theta(N)$ para actualizar *col* y $\Theta(N)$ para el recorrido: $O(N^3)$, unos $10^6$ pasos con $N = 100$.

# Traza con la muestra, *arriba* $= 1$

## Kadane sobre *col*, sin desempates

Aquí solo importa la suma, y puede ser negativa: `mejor` arranca en *col*[0], no en 0.

| `abajo` | *col* | mejor tramo de *col* |
|---|---|---|
| 1 | $[9, 2, -6, 2]$ | $9 + 2 = 11$ |
| 2 | $[5, 3, -10, 3]$ | ? |
| 3 | $[4, 11, -10, 1]$ | ? |

# Traza con la muestra, *arriba* $= 1$

## Kadane sobre *col*, sin desempates

Aquí solo importa la suma, y puede ser negativa: `mejor` arranca en *col*[0], no en 0.

| `abajo` | *col* | mejor tramo de *col* |
|---|---|---|
| 1 | $[9, 2, -6, 2]$ | $9 + 2 = 11$ |
| 2 | $[5, 3, -10, 3]$ | $5 + 3 = 8$ |
| 3 | $[4, 11, -10, 1]$ | $4 + 11 = 15$ |

## Resultado

El mejor de los diez pares de filas es 15, con `arriba` $= 1$ y `abajo` $= 3$.

# Python: maximum-sum.py

## El tramo de suma máxima

```
def mejor_tramo(v):
    mejor = v[0]
    suma = 0
    for x in v:
        suma = suma + x
        if suma > mejor:
            mejor = suma
        if suma < 0:
            suma = 0
    return mejor
```

# Python: `maximum-sum.py`

## Fijar dos filas

```
def mayor_rectangulo(m):
    n = len(m)
    mejor = m[0][0]
    for arriba in range(n):
        col = [0] * n
        for abajo in range(arriba, n):
            for c in range(n):
                col[c] = col[c] + m[abajo][c]
            s = mejor_tramo(col)
            if s > mejor:
                mejor = s
    return mejor
```

# Python: `maximum-sum.py`

## La lectura por palabras

```
def main():
    datos = sys.stdin.read().split()
    n = int(datos[0])
    m = []
    for i in range(n):
        fila = []
        for j in range(n):
            fila.append(int(datos[1 + i * n + j]))
        m.append(fila)
    print(mayor_rectangulo(m))


main()
```

# C++: maximum-sum.cpp

## El tramo de suma máxima

```
int mejor_tramo(const vector<int>& v) {
    int mejor = v[0];
    int suma = 0;
    for (int c = 0; c < (int) v.size(); c++) {
        suma = suma + v[c];
        if (suma > mejor) {
            mejor = suma;
        }
        if (suma < 0) {
            suma = 0;
        }
    }
    return mejor;
}
```

# C++: maximum-sum.cpp

## Fijar dos filas

```cpp
int mayor_rectangulo(const vector<vector<int>>& m) {
    int n = m.size();
    int mejor = m[0][0];
    for (int arriba = 0; arriba < n; arriba++) {
        vector<int> col(n, 0);
        for (int abajo = arriba; abajo < n; abajo++) {
            for (int c = 0; c < n; c++) {
                col[c] = col[c] + m[abajo][c];
            }
            int s = mejor_tramo(col);
            if (s > mejor) {
                mejor = s;
            }
        }
    }
    return mejor;
}
```

# C++: maximum-sum.cpp

## La lectura

```cpp
int main() {
    int n;
    scanf("%d", &n);
    vector<vector<int>> m(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &m[i][j]);
        }
    }
    printf("%d\n", mayor_rectangulo(m));
    return 0;
}
```

## ¿Cabe en int?

La mayor suma posible es $127 \cdot 100^2 = 1\,270\,000$.

# Trampas

## Todo negativo

Con $N = 1$ y la celda $-5$, la respuesta es $-5$: el rectángulo tiene al menos una celda. Si `mejor` arranca en 0, sale 0.

## Leer por líneas

La muestra misma reparte la matriz en líneas de 8, 5 y 3 números. Un programa que lee $N$ números por línea falla con la muestra.

## `col` sin reiniciar

*col* vuelve a ceros cada vez que cambia `arriba`; si no, arrastra filas de arriba que ya no están en el rectángulo.

# Variante para pensar

## Una matriz que no es cuadrada

La matriz tiene $f = 10$ filas y $c = 10\,000$ columnas. ¿Qué conviene fijar: dos filas o dos columnas?

## Pista

Fijar dos filas cuesta $O(f^2 c)$ y fijar dos columnas, $O(c^2 f)$. Ponga los números.

# Contenido

1 Hora 1. UVa 11078: Open Credit System

2 Hora 2. UVa 507: Jill Rides Again

3 Hora 3. UVa 108: Maximum Sum

4 Tarea del bloque

# Para el bloque 2

## Cómo se trabajan

Los tres se resuelven, se envían al juez y se revisan al comienzo del bloque siguiente. Van en orden de dificultad.

## Python en UVa

El juez corre una versión antigua de Python 3: la salida se arma con `print` y con `%`, como en las soluciones de hoy.

# Base. UVa 100: The 3n + 1 problem

## Direcciones

Enunciado: `https://onlinejudge.org/external/1/100.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=36`

## Cómo atacarlo

Un ciclo aplica la regla hasta llegar a 1 y cuenta los números que imprime; el recorrido del rango se queda con el máximo. La longitud cuenta el 1, así que la de 22 es 16. La entrada se lee hasta fin de archivo.

## La trampa

$i$ puede llegar mayor que $j$: el recorrido va del menor al mayor, pero la salida repite $i$ y $j$ en el orden en que llegaron. Los valores intermedios pasan de 10 000, así que no sirven como posición de un arreglo del tamaño de la entrada.

# Intermedio. UVa 10189: Minesweeper

## Direcciones

Enunciado: `https://onlinejudge.org/external/101/10189.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1130`

## Cómo atacarlo

Por cada celda sin mina se cuentan las minas de sus ocho vecinas, con los dos desplazamientos recorriendo $-1, 0, 1$ y la comprobación de borde. El primer número son las filas y el segundo las columnas.

## La trampa

Está en la salida: la línea en blanco va entre campos y no después del último, y las minas se conservan tal como llegaron.

# Exigente. UVa 10920: Spiral Tap

## Direcciones

Enunciado: `https://onlinejudge.org/external/109/10920.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1861`

## Cómo atacarlo

Aquí la matriz no se puede construir: con un lado de hasta 100 000 son $10^{10}$ celdas. Se trabaja por anillos, cada uno con su rango de posiciones, y adentro del anillo la posición se convierte en fila y columna con aritmética.

## La trampa

El tamaño: la posición llega hasta el lado al cuadrado y se pasa de 32 bits. La fila 1 es la de abajo y el centro de la espiral es la posición 1.

# Para cerrar el bloque

## El cuadrado de suma máxima

En una matriz de $N \times N$, encuentre el subcuadrado ($k \times k$, con cualquier $k$ de 1 a $N$) de mayor suma. Escríbalo en Python y en C++ en $O(N^3)$, y pruébelo con la muestra de UVa 108.